CLOUD FILE STORAGE SYSTEM

Prepared in the partial fulfillment of the Summer Internship Program on AWS DevOps

AT

Andhra Pradesh Skill Development Corporation (APSSDC) — Naipunyam Portal

Under the guidance of

Mrs. Bethala Sumana, APSSDC

Mr. Juttuka Lovababu, APSSDC

Submitted by

Cheerala Kona Hima Bindhu, A23126552073, ANITS, (Batch - 1)

Kuncham Varaha Venkata Supriya, A23126552090, ANITS, (Batch - 1)

Maddila Pooja Sirivarshini, A23126552093, ANITS, (Batch - 1)

Pidugu Deekshitha, A23126552108, ANITS, (Batch - 1)

Pulavarthi Gayatri, A23126552112, ANITS, (Batch - 1)

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to everyone who contributed to the successful completion of our summer internship project at the Andhra Pradesh Skill Development Corporation (APSSDC). This opportunity has been an enriching experience, and we are thankful for the support, guidance, and encouragement received throughout.

We extend our sincere regards to Mrs. Bethala Sumana and Mr. Juttuka Lovababu, our mentors, for their valuable insights, constant guidance, and unwavering support throughout the internship. Their expertise and encouragement were instrumental in shaping the direction of this project.

We would also like to thank the entire team at APSSDC for fostering a collaborative and innovative environment through the Naipunyam Portal. The knowledge sharing and feedback we received significantly contributed to the development and success of this project.

In conclusion, we are honoured to have been part of this internship program and look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Sincerely,

Cheerala Kona Hima Bindhu— himab886@gmail.com

Kuncham Varaha Venkata Supriya — kunchamvarahavenkatasupriya@gmail.com

Maddila Pooja Sirivarshini — poojamaddila21@gmail.com

Pidugu Deekshitha — deekshithapidugu2005@gmail.com

Pulavarthi Gayatri — pulavarthi.gayathri2005@gmail.com

ABSTRACT

This project presents the design and implementation of **CloudBox**, a cloud-based file storage and sharing system that enables users to securely upload, organize, access, download, and share files through a simple web application. The primary objective of the project is to provide a lightweight, secure, and user-friendly cloud storage solution by leveraging Amazon Web Services (AWS) for scalable and reliable file storage.

The system allows users to register, log in securely using JSON Web Token (JWT)-based authentication, and manage their personal files through an intuitive browser interface. Users can create folders, upload files, download stored content, rename or delete files, and generate secure shareable links for collaboration. Instead of storing files on the application server, uploaded files are transferred directly to Amazon S3 using pre-signed URLs, significantly reducing backend workload while improving upload and download performance.

The backend is developed using **Node.js** and **Express.js**, which provide RESTful APIs for user authentication, folder management, file metadata management, and secure access control. User information and file metadata are maintained in a lightweight **SQLite** database, while the actual file contents are securely stored in **Amazon S3**. Passwords are protected using hashing techniques, and JWT authentication ensures that only authorized users can access their files.

The frontend is built using **HTML, CSS, and JavaScript**, providing a responsive and easy-to-use interface for managing cloud storage. The application is deployed using **Netlify** for the frontend and **Render** for the backend, demonstrating a complete cloud-based deployment architecture suitable for real-world usage.

By combining modern web technologies with AWS cloud services, CloudBox demonstrates an efficient, secure, and scalable approach to personal cloud storage. The project eliminates many limitations of traditional local storage by enabling users to access their files from anywhere, improving data availability, enhancing collaboration through secure sharing, and reducing the risk of data loss.

TABLE OF CONTENTS

1. INTRODUCTION

2. METHODOLOGY

3. TECHNOLOGY STACK

4. SYSTEM DESIGN / ARCHITECTURE

5. IMPLEMENTATION

6. RESULTS

7. CONCLUSION

1. INTRODUCTION

In today's digital world, the amount of data generated by individuals and organizations continues to grow rapidly. Managing documents, images, videos, and other digital files efficiently has become an essential requirement. Traditional storage methods such as hard drives, USB devices, and local computer storage are often limited by device dependency, storage capacity, and the risk of hardware failure or accidental data loss. These limitations highlight the need for secure, scalable, and cloud-based storage solutions that enable users to access their files anytime and from anywhere.

Cloud computing has transformed the way data is stored and managed by providing highly available, reliable, and cost-effective storage services over the internet. Cloud storage platforms eliminate the need for users to maintain physical storage devices while ensuring that files remain accessible across multiple devices. Organizations and individuals increasingly rely on cloud storage to improve collaboration, simplify file management, and enhance data security.

The **CloudBox – Cloud File Storage System** was developed to address these challenges by providing a secure and user-friendly web application for managing digital files. The system allows users to create an account, securely log in, organize files into folders, upload and download files, delete unwanted content, rename files, and generate secure sharing links for collaboration. The application combines modern web technologies with Amazon Web Services (AWS) to provide an efficient cloud storage solution that is simple to use while maintaining strong security practices.

Unlike traditional file storage systems where uploaded files pass through the application server, CloudBox uses **Amazon S3 Pre-Signed URLs** to enable direct communication between the user's browser and Amazon S3. This approach significantly reduces the workload on the backend server, improves upload and download performance, minimizes bandwidth usage, and enhances overall scalability. The backend is responsible only for authentication, authorization, metadata management, and secure URL generation, making the application lightweight and efficient.

Security has been considered as one of the primary objectives during the development of the system. User authentication is implemented using **JSON Web Tokens (JWT)**, ensuring that only authenticated users can access their personal files. Passwords are securely encrypted before being stored in the database, preventing unauthorized access even if the database is compromised. File

metadata, including folder information and ownership details, is maintained in a **SQLite** database, while the actual file contents are stored securely in **Amazon S3**, ensuring durability and high availability.

The application follows a clear client-server architecture consisting of a browser-based frontend, a RESTful backend API developed using **Node.js** and **Express.js**, a lightweight SQLite database for metadata management, and Amazon S3 for cloud storage. The frontend provides an intuitive interface built using **HTML, CSS, and JavaScript**, allowing users to manage their files with minimal effort. The completed application has been deployed using **Netlify** for hosting the frontend and **Render** for hosting the backend API, demonstrating a complete cloud-based deployment suitable for real-world environments.

The primary objectives of this project are:

- To provide secure cloud-based storage for user files.
- To enable users to upload, download, organize, and share files through an intuitive web interface.
- To implement secure user authentication and authorization using JWT.
- To reduce backend processing by utilizing Amazon S3 Pre-Signed URLs for direct file transfers.
- To ensure reliable storage of files while maintaining lightweight metadata management.
- To demonstrate the practical integration of AWS cloud services with modern full-stack web development technologies.

This report presents the complete design, architecture, implementation, deployment, and evaluation of the CloudBox system. It explains the technologies used, the development methodology followed, the system workflow, security mechanisms, deployment process, obtained results, and potential future enhancements. The project demonstrates how cloud computing services can be effectively integrated with modern web development frameworks to build secure, scalable, and efficient cloud storage applications.

2. METHODOLOGY

The development of **CloudBox – Cloud File Storage System** followed a structured and iterative software development methodology consisting of requirements gathering, system design, implementation, testing, deployment, and continuous refinement. Each phase focused on building a secure, scalable, and user-friendly cloud storage application while following modern full-stack development practices and leveraging Amazon Web Services (AWS) for reliable file storage.

2.1. Requirements Gathering

Based on these observations, the following functional requirements were identified:

- Secure user registration and login.
- Upload files to cloud storage.
- Download stored files.
- Create, rename, and delete folders.
- Delete and rename uploaded files.
- Generate secure shareable links.
- Maintain user-specific file organization.
- Prevent unauthorized access to user files.

The non-functional requirements included scalability, security, high availability, fast file transfers, lightweight backend processing, and ease of deployment on cloud platforms.

2.2. System Design

After gathering the requirements, a client-server architecture was designed to separate the frontend, backend, database, and cloud storage components.

The system consists of four major layers:

- **Frontend Layer:** Developed using HTML, CSS, and JavaScript to provide an interactive user interface.
- **Backend Layer:** Built using Node.js and Express.js to process API requests, authenticate users, generate secure upload/download URLs, and manage application logic.
- **Database Layer:** SQLite stores user accounts, folder information, file metadata, and sharing information.
- **Cloud Storage Layer:** Amazon S3 securely stores uploaded files and provides reliable cloud-based storage.

To improve performance, uploaded files are transferred directly between the user's browser and Amazon S3 using Pre-Signed URLs. This minimizes backend workload and enables efficient handling of large files.

2.3. Implementation

The implementation phase converted the system design into a fully functional cloud storage application.Development began with creating the backend REST APIs responsible for user

authentication, folder management, metadata storage, and AWS S3 integration. JWT-based authentication was implemented to secure all protected routes and ensure that only authenticated users could access their files.The frontend was then developed to provide a clean and responsive interface where users can register, log in, upload files, manage folders, download files, rename files, delete unwanted content, and generate shareable links.AWS SDK integration was implemented to generate secure Pre-Signed URLs, enabling direct uploads and downloads between the browser and Amazon S3 without routing file contents through the backend server.

2.4. Database Management

A lightweight **SQLite** database was selected to manage application data efficiently.The database stores:

- User account information.
- Encrypted passwords.
- Folder hierarchy.
- File metadata.
- Amazon S3 object keys.
- File ownership details.
- Shared link information.

Only metadata is stored inside the database, while the actual files remain securely stored inside Amazon S3. This separation improves system performance and allows cloud storage to scale independently of the application database.

2.5. Testing

The CloudBox application was tested thoroughly to verify both functionality and security.Functional testing confirmed that:

- User registration and login work correctly.
- JWT authentication protects restricted resources.
- Files upload successfully to Amazon S3.
- Download links retrieve the correct files.
- Folder creation and deletion operate as expected.
- Rename operations update metadata correctly.
- Share links provide controlled access.
- Unauthorized users cannot access another user's files.

Additional testing verified that expired or invalid authentication tokens were rejected and that file operations were performed only after successful authorization.

2.6. Deployment

After successful testing, the application was deployed to cloud hosting platforms.The frontend was deployed on **Netlify**, providing fast and globally accessible static web hosting.The backend REST API was deployed on **Render**, which hosts the Node.js Express application and communicates

with both the SQLite database and Amazon S3 services.Amazon S3 acts as the cloud storage platform, securely storing uploaded files and serving them through time-limited Pre-Signed URLs.This deployment architecture demonstrates a complete cloud-based application where the frontend, backend, database, and storage components work together efficiently.

2.7. Version Control and Collaboration

The project source code is maintained using **Git** and **GitHub**, enabling effective version control and team collaboration throughout development.

GitHub was used to:

- Maintain complete source code history.
- Track feature development.
- Manage code updates.
- Collaborate among team members.
- Store project documentation.

Version control ensured that changes could be reviewed, tracked, and restored whenever necessary, improving the overall reliability and maintainability of the project.

2.8. Iterative Refinement

The application was continuously improved throughout development based on testing results and practical usage.

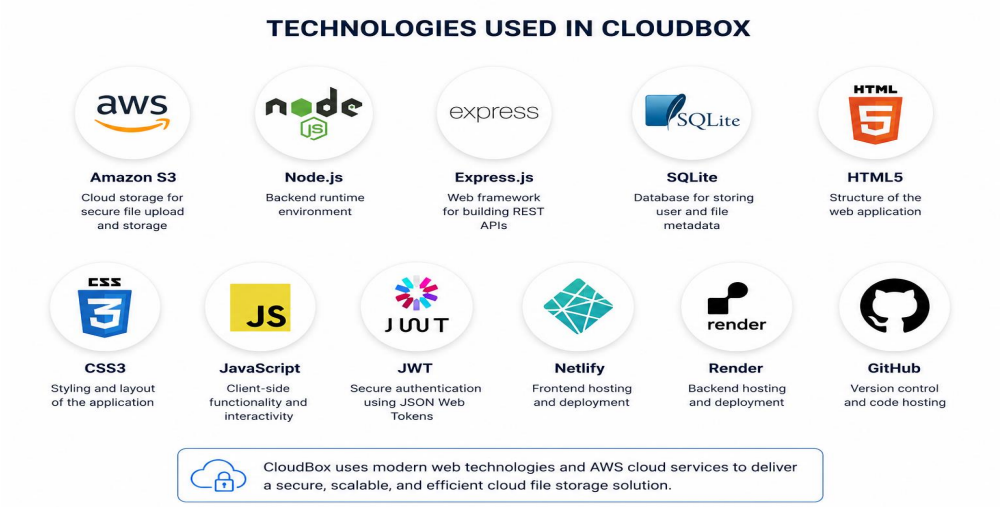
Several refinements were introduced, including:

- Improving user interface responsiveness.
- Optimizing file upload performance.
- Enhancing folder organization.
- Strengthening authentication security.
- Reducing backend workload through direct S3 transfers.
- Simplifying file sharing using secure generated links.

These improvements resulted in a lightweight, secure, and efficient cloud storage solution that offers reliable file management while utilizing modern cloud computing technologies.

3. TECHNOLOGY STACK

Campus Portal was built using a modern, cloud-native technology stack supporting event-driven architecture, role-based access, and infrastructure as code. Every component below was chosen for being a managed, serverless-first AWS or DevOps service — no component requires patching, capacity planning, or manual scaling.



3.1. Cloud Platform

Amazon Web Services (AWS) serves as the primary cloud platform for the application. AWS provides secure, scalable, and highly available cloud storage services, enabling users to upload, download, and share files without relying on local storage. Amazon S3 was chosen because of its durability, reliability, and cost-effective storage capabilities.

3.2. Backend Technologies

The backend is responsible for handling user authentication, application logic, folder management, metadata processing, and communication with AWS services.

Node.js

Node.js provides the runtime environment for executing JavaScript on the server side. Its event-driven, non-blocking architecture enables efficient handling of multiple user requests simultaneously, making it suitable for cloud-based applications.

Express.js

Express.js is used to develop RESTful APIs that manage user authentication, folder operations, file metadata, secure sharing, and AWS S3 integration. It provides a lightweight and flexible framework for backend development while simplifying routing and middleware management.

3.3. Frontend Technologies

The user interface was developed using standard web technologies to provide a responsive and user-friendly experience.

HTML5

HTML5 structures the application's webpages and provides semantic elements for forms, navigation menus, file upload interfaces, and folder organization.

CSS3

CSS3 is used to design the application's layout, colors, typography, buttons, navigation panels,and responsive user interface. It ensures consistency across different devices and screen sizes.

JavaScript

JavaScript adds dynamic functionality to the application, enabling user interactions such as file uploads, folder creation, authentication requests, file management operations, and communication with backend APIs using asynchronous requests.

3.4. Database

SQLite

SQLite serves as the lightweight relational database used for storing application metadata. Instead of storing actual file contents, the database maintains:

- User account information
- Encrypted passwords
- Folder details
- File metadata
- Amazon S3 object keys
- File ownership information
- Shared file records

Using SQLite keeps the application simple, portable, and easy to maintain while ensuring efficient metadata management.

3.5. Cloud Storage

Amazon Simple Storage Service (Amazon S3)

Amazon S3 is the primary storage service used to securely store uploaded files. Instead of storing files on the application server, CloudBox uploads files directly to Amazon S3 using Pre-Signed URLs.

The advantages of using Amazon S3 include:

- Highly durable and reliable storage
- Automatic scalability
- Secure file access
- High availability
- Reduced backend storage requirements
- Fast upload and download performance

3.6. Security Technologies

Security is one of the major components of the CloudBox system.

JSON Web Token (JWT)

JWT is used to authenticate users after successful login. Each authenticated user receives a secure token, which must accompany every protected API request. This ensures that only authorized users can perform file management operations.

Password Hashing

User passwords are encrypted before being stored in the database. Password hashing protects user credentials and significantly reduces security risks in case of unauthorized database access.

Amazon S3 Pre-Signed URLs

CloudBox uses time-limited Pre-Signed URLs to securely upload and download files directly between the user's browser and Amazon S3. These URLs automatically expire after a specified duration, preventing unauthorized access.

3.7. Deployment Platforms

The completed application has been deployed using modern cloud hosting platforms.

Netlify

The frontend application is hosted on Netlify, providing fast content delivery, HTTPS support, and reliable static website hosting. Netlify enables users to access the application through a web browser from anywhere.

Render

The backend Node.js and Express.js application is deployed on Render. It hosts the REST APIs, processes authentication requests, manages file metadata, and communicates securely with Amazon S3.

3.8. Development Tools

Several tools were used throughout the development process.

Git

Git provides distributed version control, allowing developers to track changes, maintain source code history, and collaborate efficiently.

GitHub

GitHub hosts the complete project repository, enabling collaborative development, version management, issue tracking, and project documentation.

Visual Studio Code

Visual Studio Code served as the primary development environment for writing, debugging, and testing both frontend and backend code.

AWS SDK for JavaScript

The AWS SDK enables secure communication between the backend application and Amazon S3. It is used to generate Pre-Signed URLs, upload files, retrieve objects, and manage cloud storage operations.

3.9. System Integration

The CloudBox application integrates all technologies into a unified cloud storage solution.

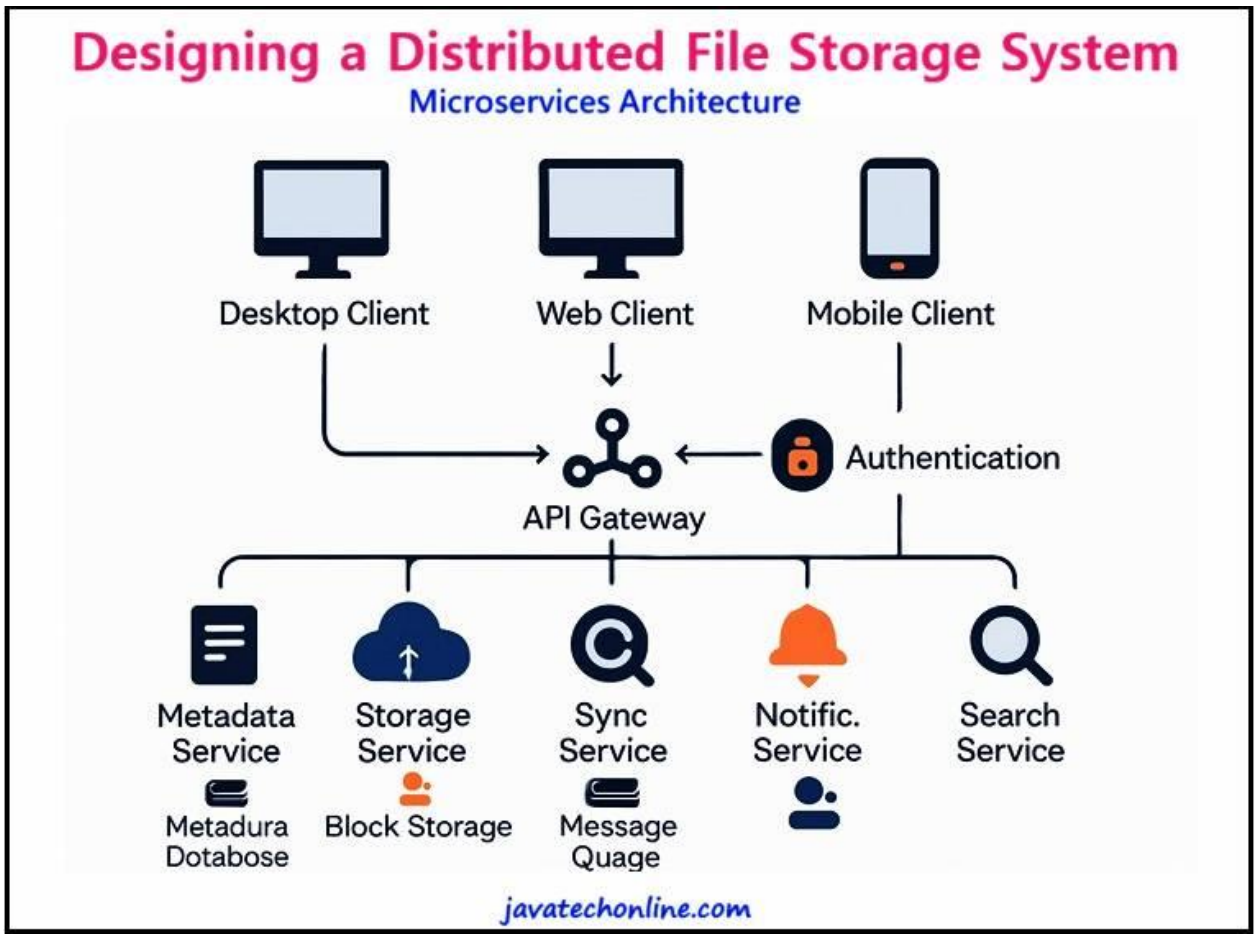
- The frontend communicates with the backend through RESTful APIs.
- The backend authenticates users using JWT.
- SQLite stores user information and metadata.
- Amazon S3 securely stores uploaded files.
- Pre-Signed URLs enable direct browser-to-S3 communication.
- Netlify hosts the frontend application.
- Render hosts the backend APIs.
- GitHub maintains the project source code.

This integrated technology stack provides a secure, scalable, and efficient cloud-based file storage system while demonstrating the practical implementation of AWS cloud services in modern web application development.

4. SYSTEM DESIGN / ARCHITECTURE

CloudBox – Cloud File Storage System follows a modern client-server architecture designed to provide secure, scalable, and efficient cloud-based file management. The application separates the frontend, backend, database, and cloud storage into independent components that communicate through RESTful APIs. This modular architecture improves maintainability, enhances security, and enables the system to scale efficiently as the number of users and files grows.

The architecture is organized into four primary layers: the Presentation Layer (Frontend), Application Layer (Backend), Data Layer (SQLite Database), and Cloud Storage Layer (Amazon S3). Together, these components deliver a complete cloud file storage solution that allows authenticated users to upload, organize, download, and share files securely.



4.1. Frontend Layer

The frontend serves as the user interface of the application and is developed using **HTML5, CSS3, and JavaScript**. It provides an intuitive browser-based interface through which users can interact with the cloud storage system.

The frontend allows users to:

- Register a new account.
- Log into the application securely.
- Create, rename, and delete folders.
- Upload files to cloud storage.
- Download previously uploaded files.
- Rename and delete files.
- Generate secure sharing links.
- Navigate through folder structures using a clean interface.

All user actions are converted into HTTP requests that are sent to the backend REST API.

4.2. Backend Layer

The backend is developed using **Node.js** and **Express.js**, providing RESTful APIs responsible for processing all application requests.

The backend performs several important functions:

- User authentication and authorization.
- JWT token generation and validation.
- Folder management.
- File metadata management.
- Share link generation.
- Communication with the SQLite database.
- Generation of Amazon S3 Pre-Signed URLs.

Unlike traditional file storage systems, the backend does not directly store uploaded files. Instead, it generates secure Pre-Signed URLs that allow the browser to communicate directly with Amazon S3. This design significantly reduces backend processing, lowers server bandwidth usage, and improves application performance.

4.3. Database Layer (SQLite)

CloudBox uses **SQLite** as its metadata database.

The database stores only application-related information rather than actual file contents.

The stored information includes:

- User details
- Encrypted passwords
- Folder hierarchy
- File names
- File paths
- Amazon S3 object keys
- Upload timestamps
- Ownership information
- Shared file records

Keeping only metadata inside the database allows file storage to remain lightweight while Amazon S3 handles large file storage independently.

4.4. Cloud Storage Layer (Amazon S3)

Amazon Simple Storage Service (**Amazon S3**) acts as the primary cloud storage solution.

Every uploaded file is stored securely inside an Amazon S3 bucket rather than on the application server.

Amazon S3 provides several advantages:

- Highly durable storage
- Automatic scalability
- High availability
- Secure object storage
- Reliable backup
- Fast file retrieval
- Cost-effective storage

The backend generates **Pre-Signed URLs**, allowing users to upload and download files directly from Amazon S3 without exposing AWS credentials.

4.5. Authentication and Authorization

Security is implemented using **JSON Web Token (JWT)** authentication.

The authentication process follows these steps:

1. A user registers with the application.
2. Passwords are securely hashed before being stored in the database.
3. During login, user credentials are verified.
4. A JWT token is generated upon successful authentication.
5. The token accompanies every protected API request.
6. The backend validates the token before performing any file operation.

This mechanism ensures that users can access only their own files and folders.

4.6. File Upload Workflow

The file upload process follows an optimized cloud architecture.

1. The user selects a file through the web interface.
2. The frontend requests an upload URL from the backend.
3. The backend authenticates the user.
4. A secure Amazon S3 Pre-Signed URL is generated.
5. The frontend uploads the file directly to Amazon S3.
6. File metadata is stored in the SQLite database.
7. The upload confirmation is displayed to the user.

This workflow eliminates unnecessary file transfers through the backend server and improves upload speed.

4.7. File Download Workflow

Downloading files follows a secure process.

1. The user selects a stored file.
2. The frontend sends a download request.
3. The backend verifies the user's JWT token.
4. A temporary Amazon S3 Pre-Signed Download URL is generated.
5. The browser downloads the file directly from Amazon S3.
6. The download link automatically expires after a specified period.

This approach protects stored files from unauthorized public access while maintaining fast download performance.

4.8. File Sharing Workflow

CloudBox allows users to share files securely.

The sharing process consists of:

- Selecting the desired file.
- Generating a unique sharing link.
- Creating a time-limited Pre-Signed URL.
- Sharing the generated link with other users.

Since the sharing URL expires automatically after a specified duration, unauthorized long-term access to files is prevented.

4.9.Security and Access Control

Multiple security mechanisms have been incorporated into the system.

The application provides:

- JWT-based authentication.
- Password hashing.
- Protected REST APIs.
- User-specific file ownership.
- Time-limited Pre-Signed URLs.
- Secure communication with AWS.
- Controlled access to cloud storage.

These security measures ensure confidentiality, integrity, and controlled access to user files.

4.10.Deployment Architecture

The application is deployed using modern cloud hosting platforms.

- **Frontend:** Netlify
- **Backend:** Render
- **Database:** SQLite
- **Cloud Storage:** Amazon S3

The deployment workflow is as follows:

User Browser → Netlify Frontend → Render Backend API → SQLite Database + Amazon S3 → Response to User

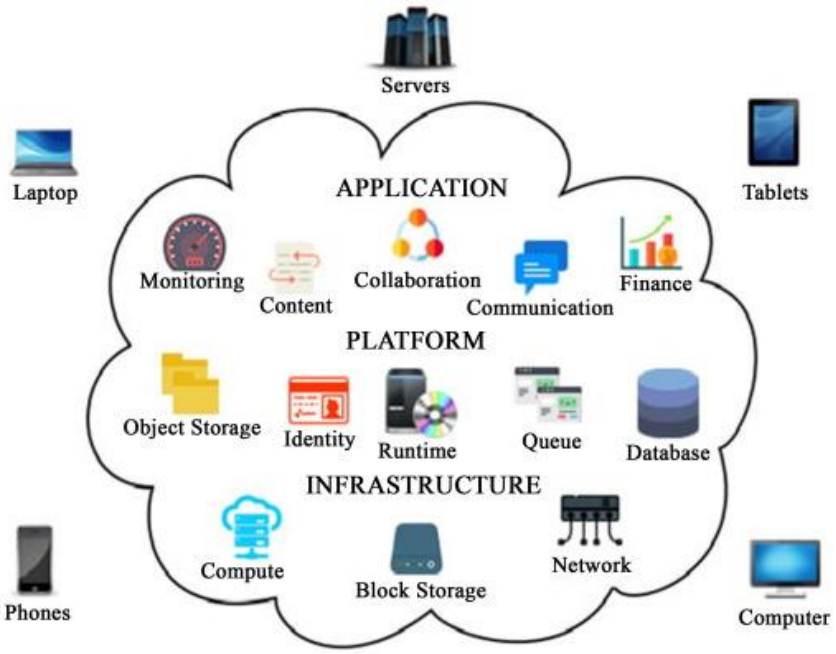
This architecture enables the frontend and backend to operate independently while providing reliable cloud storage through Amazon S3.

4.11. Overall System Workflow

The complete workflow of CloudBox can be summarized as follows:

1. User registers and logs into the application.
2. JWT authentication is performed.
3. User creates folders and manages files.
4. Files are uploaded directly to Amazon S3 using Pre-Signed URLs.
5. SQLite stores metadata related to uploaded files.
6. Users download files securely through temporary download links.
7. Share links are generated for secure collaboration.
8. The application maintains secure communication between all system components.

This layered architecture improves scalability, reduces backend processing overhead, enhances security, and provides an efficient cloud-based file storage solution suitable for personal and organizational use.



5. IMPLEMENTATION

The implementation phase transformed the proposed system architecture into a fully functional cloud-based file storage application. The project was developed using a modular approach, where the frontend, backend, database, and cloud storage components were implemented independently and later integrated into a complete system. The implementation focused on providing secure authentication, efficient file management, reliable cloud storage, and an intuitive user experience.

5.1. Project Structure

The CloudBox project is organized into separate modules to improve maintainability and simplify future enhancements.

The major project components include:

- **Frontend** – User interface developed using HTML, CSS, and JavaScript.
- **Backend** – REST APIs developed using Node.js and Express.js.
- **Database** – SQLite database for storing user accounts and file metadata.
- **Cloud Storage** – Amazon S3 for storing uploaded files.
- **Deployment** – Netlify for frontend hosting and Render for backend deployment.

This modular structure allows each component to function independently while communicating through secure APIs.

5.2. User Registration and Authentication

The first module implemented was the user authentication system.

Users can create an account by providing the required registration details. During registration, passwords are securely encrypted using password hashing techniques before being stored in the SQLite database.

When a registered user logs in:

- User credentials are validated.
- Password hashes are verified.
- A JSON Web Token (JWT) is generated.
- The token is returned to the client.
- All subsequent API requests include the JWT for authentication.

This mechanism ensures that only authenticated users can access protected resources and perform file management operations.

5.3. Folder Management

CloudBox allows users to organize files into folders for better storage management.

The folder management module supports:

- Creating new folders
- Renaming existing folders
- Deleting folders
- Viewing folder contents
- Organizing files hierarchically

Folder information is maintained within the SQLite database, allowing users to manage their personal storage space efficiently.

5.4. File Upload Implementation

The file upload functionality is one of the core features of the application.

The upload process follows these steps:

1. The user selects a file from the browser.
2. The frontend requests an upload URL from the backend.
3. The backend authenticates the user using JWT.
4. A secure Amazon S3 Pre-Signed URL is generated.
5. The browser uploads the file directly to Amazon S3.
6. File metadata is stored in the SQLite database.
7. The user receives a confirmation after successful upload.

Using Pre-Signed URLs eliminates unnecessary file transfers through the backend server, resulting in faster uploads and reduced server resource consumption.

5.5. File Download Implementation

The download module enables users to retrieve stored files securely.

The implementation process includes:

- User authentication.
- Metadata verification.
- Generation of a temporary Pre-Signed Download URL.
- Direct file download from Amazon S3.

Since download URLs expire automatically after a limited time, unauthorized users cannot repeatedly access stored files using old links.

5.6. File Management Operations

CloudBox provides complete file management functionality.

Implemented operations include:

- Viewing uploaded files.
- Renaming files.
- Deleting unwanted files.
- Moving files between folders.
- Viewing upload timestamps.
- Displaying file size information.

Whenever a file operation is performed, the corresponding metadata stored in SQLite is updated to maintain consistency with Amazon S3.

5.7.File Sharing Implementation

The secure sharing module enables users to share uploaded files with others.

The sharing process includes:

- Selecting a file.
- Generating a secure sharing link.
- Creating a time-limited Amazon S3 Pre-Signed URL.
- Sharing the generated URL with authorized users.

The shared link automatically expires after a predefined duration, improving security while enabling convenient collaboration.

5.8. Amazon S3 Integration

Amazon S3 serves as the cloud storage service for the application.

The AWS SDK is integrated into the backend to perform cloud storage operations.

The backend performs the following tasks:

- Generate upload URLs.
- Generate download URLs.
- Store file object information.
- Manage file paths.

- Verify storage operations.

Actual file contents remain stored inside Amazon S3, while only metadata is maintained in SQLite.

This architecture significantly improves scalability and storage efficiency.

5.9.Database Implementation

SQLite manages all application-related information.

The database contains tables for:

- Registered users
- Folder information
- Uploaded file metadata
- File ownership
- Shared links

Each uploaded file is associated with its owner, upload time, folder location, and Amazon S3 object key.

Separating metadata from actual file storage improves application performance and simplifies database management.

5.10.Frontend Implementation

The frontend provides an interactive interface for users.

Major interface components include:

- Login Page
- Registration Page
- Dashboard
- Folder Management
- File Upload Window
- File List
- Share File Interface
- Download Section

JavaScript communicates with backend APIs using asynchronous HTTP requests, allowing users to perform operations without refreshing the webpage.

The responsive interface provides a smooth user experience across desktop and mobile devices.

5.11. Deployment

After implementation and testing, the application was deployed using cloud hosting services.

Deployment architecture includes:

- **Frontend Deployment:** Netlify
- **Backend Deployment:** Render
- **Cloud Storage:** Amazon S3

The frontend communicates securely with the deployed backend APIs, which in turn interact with SQLite and Amazon S3 to process user requests.

This deployment demonstrates a complete cloud-based architecture suitable for real-world applications.

5.12. Security Implementation

Several security mechanisms were incorporated throughout the application.

These include:

- JWT-based authentication.
- Password hashing before database storage.
- Protected REST APIs.
- User-specific authorization.
- Time-limited Amazon S3 Pre-Signed URLs.
- Secure communication between frontend and backend.

Together, these mechanisms ensure that users can access only their own files while protecting stored data against unauthorized access.

5.13.Overall Implementation Outcome

The successful implementation of CloudBox resulted in a fully functional cloud storage system capable of securely storing, organizing, sharing, and retrieving files through a web browser.

The integration of modern web technologies with Amazon Web Services provides a scalable, secure, and efficient cloud storage solution. By combining HTML, CSS, JavaScript, Node.js, Express.js, SQLite, JWT authentication, Amazon S3, Netlify, and Render, the project demonstrates a practical implementation of cloud computing concepts and full-stack web development suitable for both academic learning and real-world deployment.

6.RESULTS

The implementation of **CloudBox – Cloud File Storage System** resulted in a fully functional cloud-based application capable of securely storing, managing, downloading, and sharing files through a web browser. The application was successfully tested across its major functional modules, including authentication, folder management, file upload, download, sharing, and deployment. The integration of Amazon S3 with modern web technologies demonstrated the effectiveness of cloud storage in delivering a reliable, scalable, and user-friendly file management solution.

6.1.User Authentication

The authentication module was successfully tested for both user registration and login.

The following functionalities worked as expected:

- New user registration.
- Secure password storage using hashing.
- Successful user login.
- JWT token generation after authentication.
- Protected API access using JWT.
- Session validation for authorized users.

Unauthorized users attempting to access protected resources without a valid token were denied access, confirming that the authentication mechanism functions correctly.

6.2. File Upload Performance

The file upload functionality was successfully implemented and tested.

During testing:

- Files of different formats were uploaded successfully.
- Upload requests generated secure Amazon S3 Pre-Signed URLs.
- Files were stored directly in Amazon S3.
- Metadata was recorded correctly in the SQLite database.
- Upload confirmation was displayed after successful completion.

Direct browser-to-S3 uploads reduced backend workload and provided faster upload performance.

6.3. File Download Validation

The download functionality operated successfully.

Testing verified that:

- Stored files could be retrieved correctly.
- Temporary download links were generated securely.
- Download URLs expired automatically after the configured duration.
- Unauthorized download attempts were rejected.

This confirms that the application provides secure and efficient file retrieval while protecting cloud-stored resources.

6.4. Folder and File Management

The folder management module was validated through multiple test cases.

Users were able to:

- Create folders.
- Rename folders.
- Delete folders.
- Upload files into folders.
- Rename uploaded files.
- Delete unwanted files.
- View organized file structures.

All changes were reflected correctly within the SQLite database, maintaining synchronization between metadata and Amazon S3 storage.

6.5. Secure File Sharing

The sharing feature was tested successfully.

The application allowed users to:

- Generate secure sharing links.
- Share uploaded files.
- Access files using generated links.
- Restrict access through time-limited Pre-Signed URLs.

The sharing mechanism demonstrated a secure approach to collaboration while preventing permanent public access to stored files.

6.6.System Performance

CloudBox performed efficiently throughout testing.

The following observations were recorded:

- Fast user authentication.
- Responsive user interface.
- Reduced backend processing.
- Smooth communication with Amazon S3.
- Quick upload and download operations.
- Efficient metadata retrieval from SQLite.

Since actual file contents are transferred directly between the browser and Amazon S3, the backend server experiences significantly lower bandwidth usage compared to traditional file upload systems.

6.7. Security Evaluation

Several security features were evaluated during testing.

The application successfully demonstrated:

- JWT-based authentication.
- Password hashing.
- User-specific authorization.
- Protected REST APIs.
- Secure cloud communication.
- Time-limited Pre-Signed URLs.
- Controlled access to uploaded files.

These mechanisms provide strong protection against unauthorized access while ensuring that users can access only their own files.

6.8. Deployment Results

The application was successfully deployed on cloud hosting platforms.

Deployment components include:

- **Frontend:** Netlify
- **Backend:** Render
- **Cloud Storage:** Amazon S3

The deployed application successfully handled:

- User registration.
- User login.
- Folder creation.
- File uploads.
- File downloads.
- Secure file sharing.

The successful deployment demonstrates that the system is suitable for real-world cloud-based environments.

6.9. Operational Benefits

The completed CloudBox application offers several practical advantages.

These include:

- Cloud-based file accessibility from anywhere.
- Secure file storage.
- Efficient file organization.
- Lightweight backend architecture.
- Scalable cloud storage.
- Reduced server maintenance.
- Easy deployment.
- Secure collaboration through file sharing.

These benefits make CloudBox suitable for students, professionals, and organizations requiring reliable cloud-based file management.

6.10.Limitations

Although the project successfully meets its objectives, certain limitations remain.

Current limitations include:

- SQLite is suitable for small to medium-scale applications but may not be ideal for large enterprise deployments.
- File versioning is not currently supported.
- No recycle bin functionality for deleted files.
- Limited file preview support within the browser.
- No advanced search or filtering options.

- Storage quotas are not enforced for individual users.

These limitations provide opportunities for future improvements.

6.11.Cost and Operational Efficiency

CloudBox leverages cloud technologies to reduce infrastructure costs and improve operational efficiency.

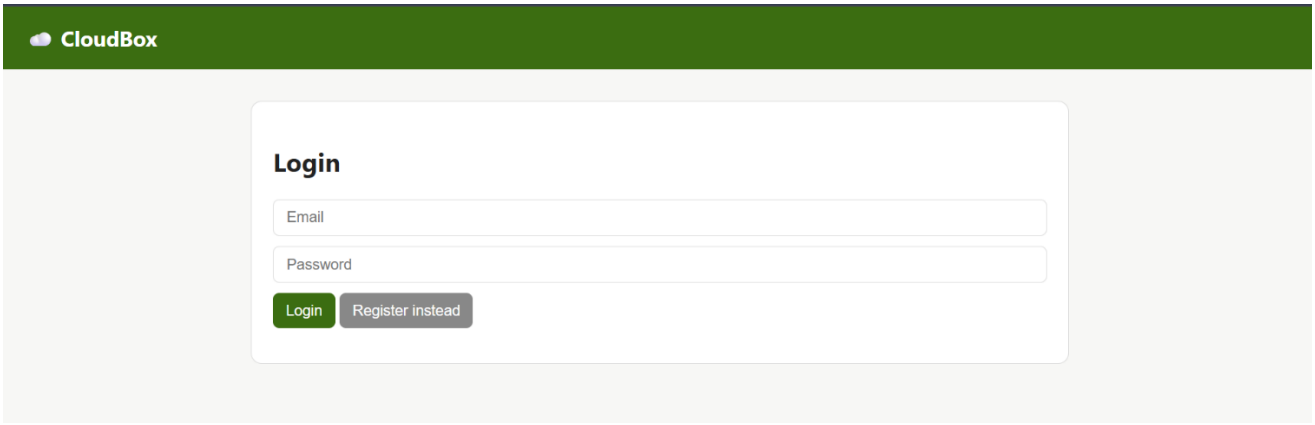
The use of Amazon S3 provides:

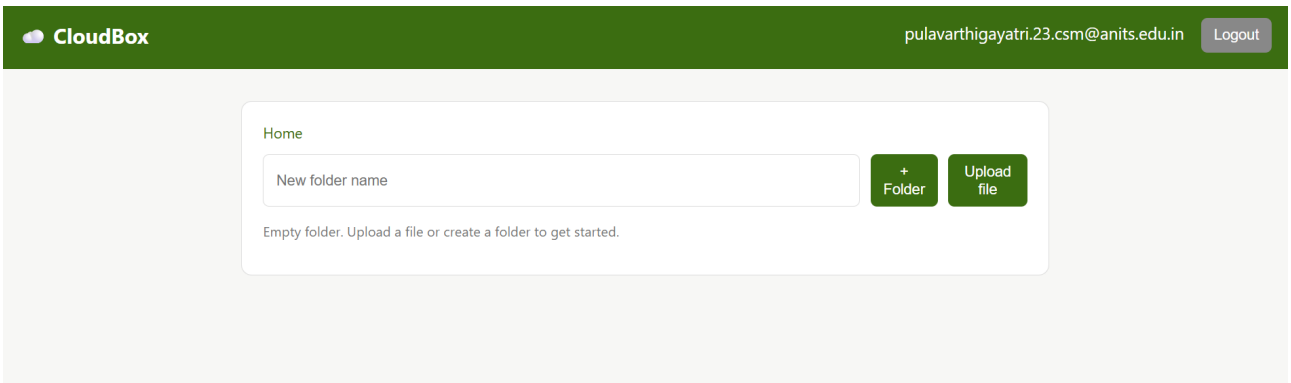
- Pay-as-you-use cloud storage.
- High durability.
- Automatic scalability.
- Minimal infrastructure maintenance.

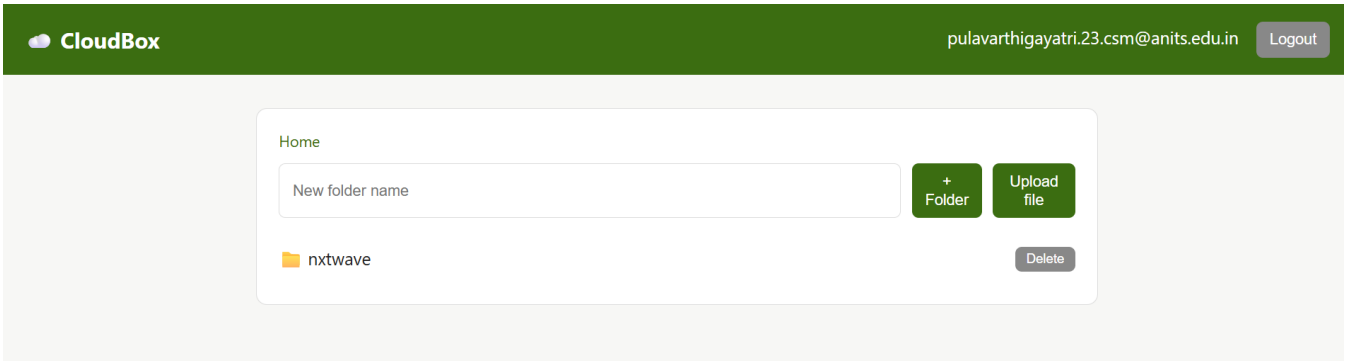
Deploying the frontend on Netlify and the backend on Render further reduces operational overhead while providing reliable application hosting.

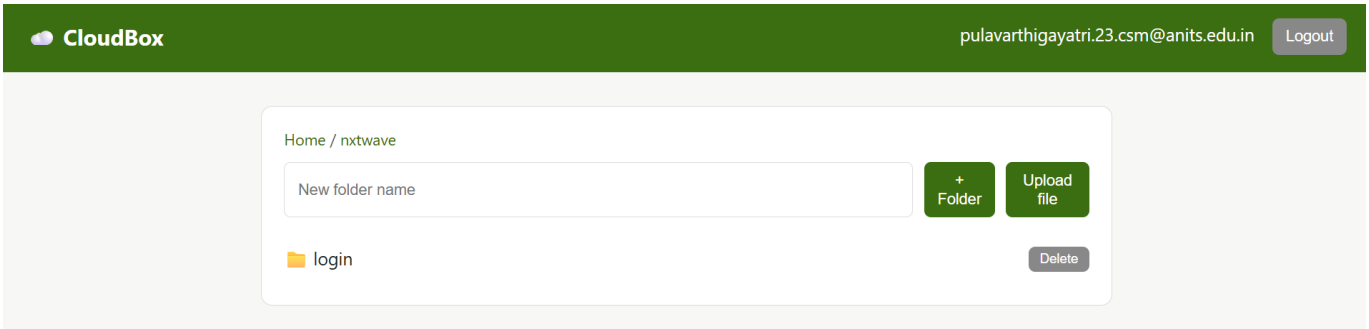
6.12.Application Screenshots

The following screenshots demonstrate the successful implementation of CloudBox.









These screenshots verify that the implemented features operate successfully and demonstrate the complete functionality of the CloudBox application.

6.13.Overall Result

The overall outcome of the project confirms that CloudBox successfully achieves its primary objectives of providing secure cloud-based file storage, efficient file management, reliable cloud deployment, and secure file sharing. The combination of modern web technologies with Amazon Web Services results in a scalable, user-friendly, and secure cloud storage platform suitable for academic demonstration and real-world applications.

7.CONCLUSION

The successful design, development, and deployment of **CloudBox – Cloud File Storage System** demonstrate the practical application of cloud computing technologies in building secure, scalable, and user-friendly file storage solutions. By integrating modern web development technologies with Amazon Web Services (AWS), the project provides an efficient platform for storing, organizing, managing, downloading, and sharing digital files over the internet.

The project effectively combines **HTML, CSS, JavaScript, Node.js, Express.js, SQLite, Amazon S3, JSON Web Token (JWT) authentication, Netlify, and Render** to create a complete full-stack cloud application. The use of Amazon S3 as the primary storage service eliminates the limitations of traditional local storage by providing highly durable, scalable, and reliable cloud storage. The implementation of Pre-Signed URLs further enhances system efficiency by allowing direct communication between the user's browser and Amazon S3, reducing backend processing and improving upload and download performance.

Security was a major consideration throughout the development process. Password hashing, JWT-based authentication, protected REST APIs, and secure Pre-Signed URLs ensure that only authorized users can access their files while protecting stored data from unauthorized access. The separation of file metadata and actual file storage further improves system organization, maintainability, and scalability.

From a software engineering perspective, the project provided valuable practical experience in full-stack web development, RESTful API design, cloud storage integration, authentication mechanisms, database management, and cloud deployment. Developing the application strengthened understanding of modern application architecture, cloud computing concepts, secure web development practices, and AWS services commonly used in industry.

CloudBox successfully achieves the objectives defined at the beginning of the project by providing:

- Secure user authentication and authorization.
- Cloud-based file storage using Amazon S3.
- Efficient file upload and download functionality.

- Folder-based file organization.
- Secure file sharing through temporary links.
- Responsive web interface.
- Reliable cloud deployment using Netlify and Render.
- Lightweight and scalable application architecture.
-

Although the current implementation satisfies the primary requirements, several enhancements can further improve the system in future versions.

These include:

- Implementing file versioning and recovery.
- Adding a recycle bin for deleted files.
- Supporting drag-and-drop file uploads.
- Providing in-browser previews for documents, images, videos, and PDF files.
- Implementing advanced search and filtering capabilities.
- Introducing storage quota management for users.
- Enabling multi-user collaboration and shared workspaces.
- Integrating multi-factor authentication (MFA) for enhanced security.
- Migrating from SQLite to a cloud-hosted relational database such as Amazon RDS or PostgreSQL for large-scale deployments.
- Adding activity logs, notifications, and analytics dashboards.

In conclusion, **CloudBox – Cloud File Storage System** serves as an effective demonstration of how modern cloud services and full-stack web technologies can be integrated to build secure, scalable, and maintainable applications. The project successfully addresses common challenges associated with traditional file storage systems while showcasing industry-standard development practices. It provides a strong foundation for future enhancements and can be extended into an enterprise-grade cloud storage platform suitable for both personal and organizational use.